

C++

Programming Language

a complete beginner's handwritten notebook

Features • How C++ Works • Applications • Roadmap

Installing C++ • Syntax • Data Types • Operators

Loops • Functions • Input / Output • Pointers & OOP

Dependencies & Building An App • Advantages

Phumeh Mjoli

Portfolio: <https://uphume.netlify.app/Portfolio>

1. What Is C++? & Its Features

C++ is a **compiled, statically-typed, general-purpose language** created as an extension of C, adding Object-Oriented Programming on top. "Compiled" means your whole program is translated into machine code by a compiler BEFORE it ever runs – the CPU runs your program directly, with nothing standing in between.

Why does this matter? Because that direct-to-machine-code translation is exactly why C++ is one of the fastest languages around, and why it's the language of choice whenever raw speed and control over memory really matter – game engines, operating systems, embedded devices.

Key Features (and WHY each one is useful)

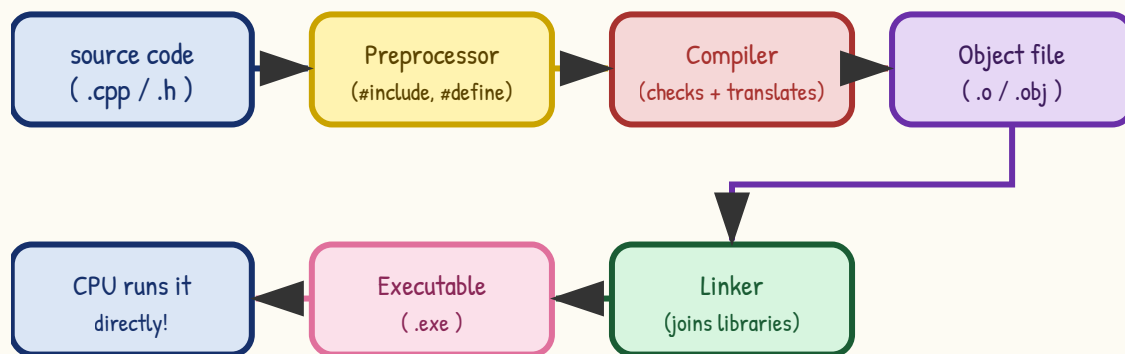
- **Compiled language** – translated straight to machine code, so it runs extremely fast. Trade-off: you must recompile every time you change the code.
- **Statically typed** – you must declare a variable's type (`int age = 25;`) and it can never silently change type. This catches many bugs BEFORE the program even runs.
- **Manual memory management** – you control exactly when memory is allocated (`new`) and freed (`delete`). Powerful, but you are responsible for cleaning up after yourself.
- **Object-Oriented** – supports classes, inheritance, polymorphism, so you can model real-world things as objects.
- **Multi-paradigm** – procedural, object-oriented AND generic (templates) programming are all supported, so you pick whatever style fits.

- **Rich Standard Library (STL)** – ready-made containers (vectors, maps) and algorithms (sorting, searching) so you don't reinvent basic data structures.
- **Portable** – write once, recompile for the target platform (Windows/macOS/Linux each need their own compiled binary, unlike Python's "run anywhere" bytecode).
- **Close to hardware** – pointers and references let you work directly with memory addresses, which is essential for systems programming.

***Quick note:** Static typing + manual memory management are the two features that make C++ harder to learn than Python, but they are also exactly why C++ gives you far more control and speed. That trade-off (safety-net vs raw control) is the single biggest thing to understand about why C++ gets chosen for performance-critical projects.*

2. How C++ Works (The Build Pipeline)

Unlike an interpreted language, C++ needs a "build" step before it can run. Understanding this pipeline explains why you must compile every time you edit your code.



source → preprocessor → compiler → object file → linker → executable → CPU

Step by step

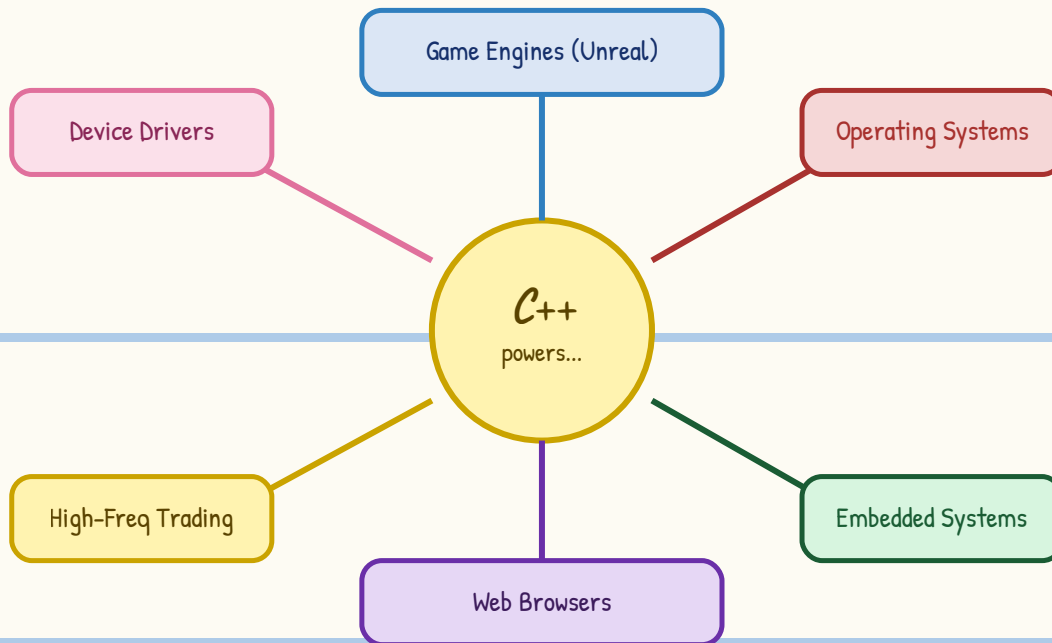
1. The **preprocessor** handles lines starting with `#`, like copy-pasting the contents of `#include <iostream>` straight into your file.
2. The **compiler** checks your syntax/types and translates the code into an **object file** of machine instructions (one object file per `.cpp` file).
3. The **linker** stitches all your object files together with any libraries you used (like the standard library) into one final **executable**.
4. The executable is pure machine code – the CPU runs it directly, with no interpreter or virtual machine in the way.

Why should you care? This is exactly why a C++ program typically starts up and runs faster than an equivalent Python script – all

the "translation work" happened once, ahead of time, instead of every single run.

3. Common Applications of C++

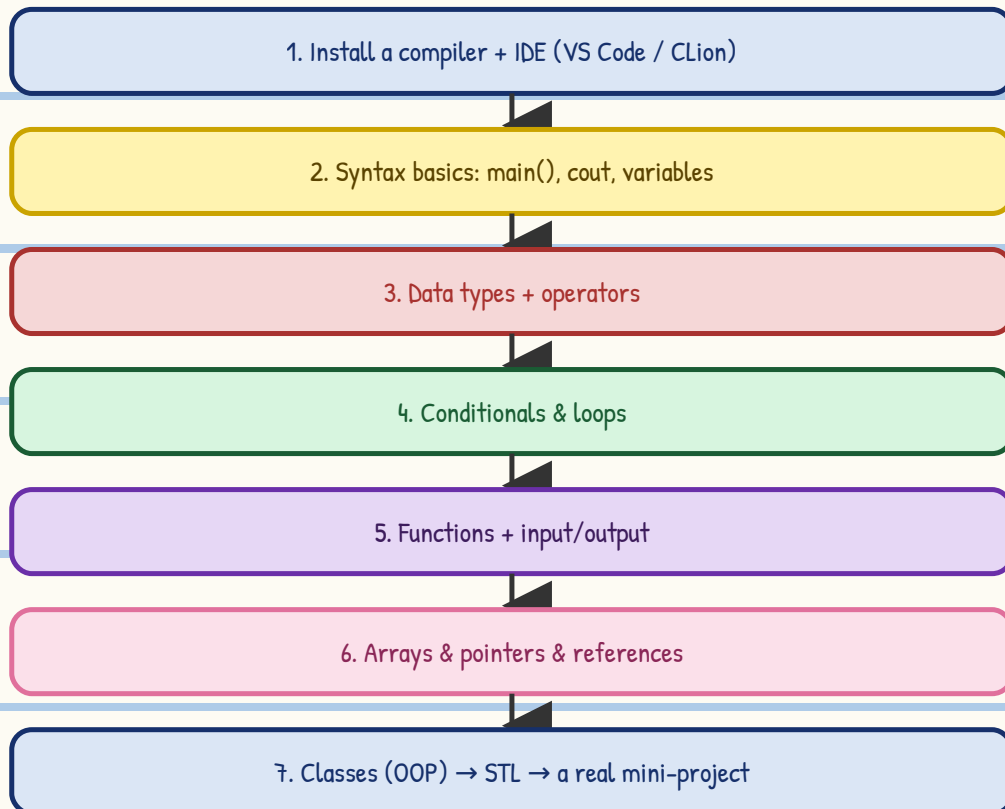
C++ is chosen whenever a program needs to be extremely fast and/or needs tight control over hardware and memory.



Field	Why C++ fits
Game Engines	Needs to render frames dozens of times per second – no room for a slow interpreter
Operating Systems	Windows, parts of Linux/macOS are written in C/C++ for direct hardware control
Embedded Systems	Microcontrollers have tiny memory/CPU – C++ gives full control with little overhead
Web Browsers	Chrome, Firefox's rendering engines are C++ for speed
High-Frequency Trading	Every microsecond of latency costs money
Device Drivers	Talk directly to hardware, needs low-level memory access

4. C++ Roadmap For Beginners

Learn in this order – C++ builds concepts on top of each other more than most languages.



***Why in this order?** Pointers only make sense once you understand variables and memory; classes only make sense once you're comfortable with functions. Skipping ahead to pointers/OOP too early is the #1 reason beginners find C++ "scary".*

5. Installing C++

C++ has no single official installer like Python – you install a **compiler** (the program that turns your code into an executable) plus an editor/IDE.

On Windows

1. **Easiest way:** install **Visual Studio Community** (free) from visualstudio.microsoft.com and tick the "Desktop development with C++" workload during setup – this gives you the MSVC compiler, debugger and editor all in one.
2. **Lighter alternative:** install **VS Code** + the **MinGW-w64** compiler (a Windows build of GCC), then add MinGW's **bin** folder to your PATH.
3. Confirm it worked by opening a terminal and typing `g++ --version` (MinGW) or using Visual Studio's built-in "Build" button.

On macOS

1. Open Terminal and run:

```
xcode-select --install
```

This installs Apple's Command Line Tools, which includes the **Clang** compiler – that's all most people need.

2. Confirm with `clang++ --version`.
3. Then install **VS Code** (or Xcode itself) as your editor, and add the C/C++ extension for syntax highlighting and debugging.

Additional notes:

- *Compiling from the terminal looks like:*

```
g++ myprogram.cpp -o myprogram
./myprogram      # run it (Mac/Linux)
myprogram.exe    # run it (Windows)
```

- *The three most common compilers are **GCC** (g++), **Clang** (clang++) and **MSVC** (Microsoft's, bundled with Visual Studio).*
- *Make sure you're targeting a modern C++ standard, e.g. add the flag `-std=c++17` when compiling, so you get access to newer language features.*

6. C++ Syntax Basics

The skeleton of every C++ program

```
#include <iostream>    // gives us access to cin/cout
using namespace std;   // so we can write "cout" instead of "std::cout"

int main(){
    cout << "Hello, World!" << endl;
    return 0;          // tells the OS the program finished OK
}
```

Why the structure looks like this: every C++ program needs exactly one `main()` function – that's where the program starts running. `#include` lines pull in code other people already wrote (like input/output tools) so you don't have to write everything from scratch.

Curly braces & semicolons

Unlike Python, C++ uses `{ }` to group blocks of code, and every statement ends with a `;`. Indentation is just for humans to read – the compiler doesn't care about it.

Why this exists: because whitespace isn't meaningful, C++ needs an explicit symbol to know "this statement has ended" – that's the semicolon's whole job. Forgetting one is the #1 beginner compile error.

Comments

```
// this is a single-line comment  
/* this is a  
multi-line comment */
```

Variables

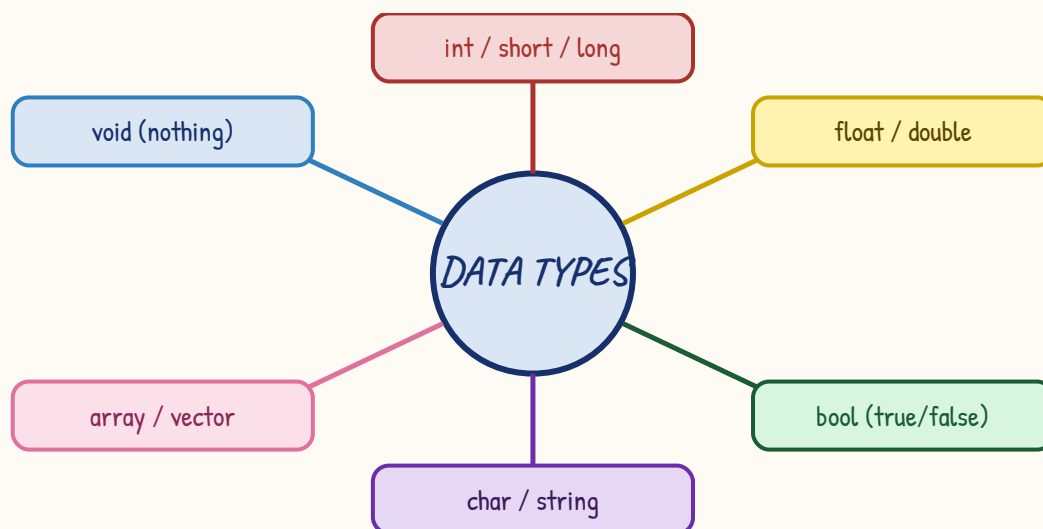
Unlike Python, you MUST state the type when creating a variable:

```
int age = 21;  
double price = 9.99;  
char grade = 'A';  
bool isStudent = true;
```

Rule of thumb: variable names are usually `camelCase` in C++, can't start with a number, and can't be a reserved word like `int` or `class`.

7. C++ Data Types

Because C++ is statically typed, the type you choose decides exactly how much memory is reserved and what operations are legal – this is stricter (and faster) than Python's flexible types.



Type	Example	Typical size	When you use it
<code>int</code>	<code>int x = 10;</code>	4 bytes	whole numbers, counting
<code>float</code>	<code>float x = 3.14f;</code>	4 bytes	decimals when memory is tight
<code>double</code>	<code>double x = 3.14159;</code>	8 bytes	decimals needing more precision (the default choice)
<code>char</code>	<code>char x = 'A';</code>	1 byte	a single character
<code>bool</code>	<code>bool x = true;</code>	1 byte	yes/no, on/off decisions
<code>string</code>	<code>string x = "hi";</code>	varies	text (needs <code>#include <string></code>)
<code>array</code>	<code>int a[5];</code>	fixed	a fixed-size list of the same type
<code>vector</code>	<code>vector<int> v;</code>	grows	a resizable list (the STL's answer to Python's list)

<code>void</code>	<code>void greet()</code>	—	"this function returns nothing"
-------------------	---------------------------	---	---------------------------------

How do you know which one to use? Need decimals? Default to `double` (more precise than `float`). Need a growable list? Use `vector` instead of a raw array – it manages its own memory safely for you.

8. C++ Operators

Arithmetic

Operator	Meaning	Example
<code>+ - * /</code>	add, subtract, multiply, divide	<code>7 / 2 → 3</code> (int division truncates!)
<code>%</code>	modulus (remainder)	<code>7 % 2 → 1</code>
<code>++ / --</code>	increment / decrement	<code>x++</code> ; same as <code>x = x + 1;</code>

Gotcha to remember: dividing two `int`s always gives an `int` back – `7 / 2` gives `3`, not `3.5`! Make at least one side a `double` if you need a decimal result.

Comparison (returns bool)

Operator	Meaning
<code>==</code>	equal to
<code>!=</code>	not equal to
<code>> < >= <=</code>	greater / less / greater-or-equal / less-or-equal

Logical

Operator	Meaning
<code>&&</code>	AND – both sides must be true
<code> </code>	OR – at least one side must be true



NOT – flips true/false

Assignment

= stores a value. Shortcuts like `x += 1;` mean "add 1 to x and save it back into x".

Bonus: C++ also has bitwise operators (`& | ^ ~ << >>`) that work directly on the binary bits of a number – used in low-level/embedded programming, not needed for everyday beginner code.

9. Loops In C++

Just like in any language, a loop repeats a block of code. Use one the moment you catch yourself thinking "I need to do this same thing many times".

for loop – when you know how many times

```
for(int i = 0; i < 5; i++){  
    cout << i << endl; // prints 0,1,2,3,4  
}
```

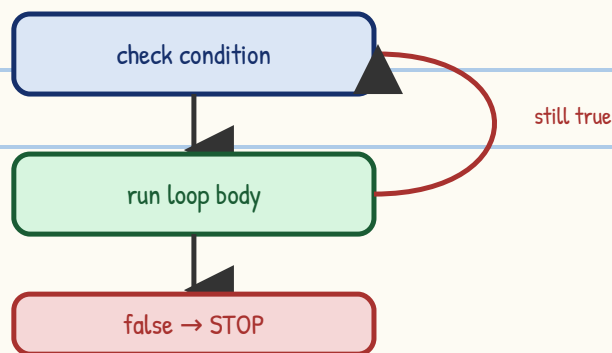
The three parts are: **start; condition; what happens after each loop**.

while loop – when you don't know how many times

```
int count = 0;  
while(count < 3){  
    cout << "counting: " << count << endl;  
    count++;  
}
```

do-while – runs at least once

```
int choice;  
do{  
    cout << "Enter 1 to continue, 0 to stop: ";  
    cin >> choice;  
} while(choice != 0);
```



*for vs while vs do-while: know the exact count or looping over an array → **for**. Looping until a condition changes → **while**. Need the body to run at least once no matter what (like a menu) → **do-while**.*

10. Functions In C++

A function is a named, reusable block of code. In C++ you must state the type it returns AND the type of every parameter – the compiler checks these match everywhere the function is used.

```
int add(int a, int b){ // "int" = what this function returns
    return a + b;
}

int main(){
    int result = add(3, 4); // calling the function
    cout << result << endl;
    return 0;
}
```



Pass by value vs pass by reference

```
void addOne(int x){ x = x + 1; } // pass by VALUE: original is untouched
void addOneRef(int &x){ x = x + 1; } // pass by REFERENCE: original DOES change
```

Why this matters: by default C++ copies the value you pass in, so changes inside the function don't affect the original variable. Adding `&` passes the REAL variable in, which is faster for big

data and lets the function actually modify it – a concept Python hides from you but C++ makes explicit.

Why use functions?

- *Avoid repeating yourself – fix a bug in one place, not ten.*
- *Break big problems into small pieces – easier to test and reason about.*

11. Input And Output In C++

`cout` sends output *TO* the screen. `cin` reads input *FROM* the keyboard. Both live in `<iostream>`.

Output – cout

```
cout << "Hello!" << endl;  
int score = 95;  
cout << "Score: " << score << endl;
```

The `<<` symbol is called the "insertion operator" – think of it as "insert this into the output stream".

Input – cin

```
string name;  
cout << "What is your name? ";  
cin >> name;  
cout << "Hi " << name << endl;  
  
int age;  
cout << "How old are you? ";  
cin >> age;           // cin already knows to read a number  
cout << "Next year you'll be " << (age + 1) << endl;
```

Difference from Python: because C++ is statically typed, `cin` automatically converts the typed text into whatever type your

variable already is – you don't need Python's manual `int()` conversion step.

Gotcha: plain `cin >>` stops reading at the first space, so it can't capture "John Smith" in one go. Use `getline(cin, name);` instead when you need a full line of text.

12. Pointers, References & Memory

This is the concept that makes C++ different from most beginner-friendly languages: you can see and control the actual memory address where a value lives.

```
int age = 21;
int* p = &age;    // p now STORES the memory address of age
cout << *p;       // "*p" means "the value stored at that address" -> 21
*p = 22;          // changes age itself, through the pointer!
```



a pointer doesn't hold the value itself - it holds the ADDRESS where the value lives

References – a safer alias

```
int age = 21;
int &r = age; // "r" is just another name for the SAME variable
r = 22;       // this also changes age
```

Manual memory: new / delete

```
int* arr = new int[5]; // manually reserve memory on the "heap"
// ... use arr ...
delete[] arr;          // YOU must free it, or it's a "memory leak"
```

Why you need to know this: Python quietly manages memory for you in the background (garbage collection). C++ gives you that

control back – more power, but forgetting a `delete` causes a memory leak, and using a pointer after its memory was freed causes a crash ("dangling pointer"). This is the single biggest source of C++ bugs, which is why modern C++ encourages "smart pointers" (`unique_ptr`, `shared_ptr`) that clean up after themselves automatically.

13. Classes & Object-Oriented Programming

A **class** is a blueprint for creating objects that bundle together data (attributes) and behaviour (methods) – the core idea behind Object-Oriented Programming.

```
class Dog{
    public:
        string name;    // attribute (data)
        int age;

        void bark(){    // method (behaviour)
            cout << name << " says Woof!" << endl;
        }
};

int main(){
    Dog myDog;        // create an OBJECT from the class
    myDog.name = "Rex";
    myDog.age = 3;
    myDog.bark();      // "Rex says Woof!"
}
```

Constructors – set up an object automatically

```
class Dog{
    public:
        string name;
        Dog(string n){ name = n; } // constructor: runs when the object is created
}
```

```
};  
Dog myDog("Rex"); // name is set immediately
```

***Why use classes?** They let you model real-world things (a Dog, a BankAccount, a Player) as one neat package instead of scattering loose variables and functions everywhere – and they let you reuse and extend behaviour through **inheritance** (e.g. a **Puppy** class could reuse everything from **Dog**).*

***public vs private:** marking data **private** hides it from outside code, forcing people to go through your methods to use it safely – this is called **encapsulation**, and it's a key OOP idea for protecting your data from misuse.*

14. Dependencies & Building An App

Bigger C++ projects span multiple files and rely on external libraries, so you need a system to organise and build them all together.

Header files vs source files

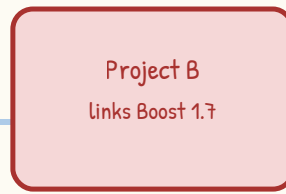
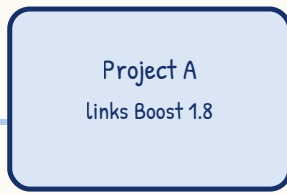
Code is usually split: a `.h` file declares WHAT a function/class looks like, and a `.cpp` file defines HOW it actually works – other files just `#include` the header to use it, without needing to see the implementation.

Static vs dynamic libraries

Static library (.lib / .a)	Dynamic library (.dll / .so)
Copied directly INTO your executable at build time	Loaded separately WHEN the program runs
Bigger executable, no extra files needed to distribute	Smaller executable, but the .dll/.so must ship alongside it

Build systems – CMake

Typing `g++ file1.cpp file2.cpp -o app` by hand gets painful fast. **CMake** is the most common tool that describes your whole project (files, libraries, settings) in one `CMakeLists.txt` file, then generates the right build commands for you on any platform.



a package manager (vcpkg / Conan) keeps each project's exact library versions separate

Package managers

vcpkg and **Conan** are C++'s answer to Python's **pip** – they download and build external libraries for you instead of you hunting for them manually online.

***Why this matters:** "building an app" in C++ is really about organising many small compiled pieces (your code + libraries) into one coherent program – CMake + a package manager is what lets a project scale from one file to thousands without becoming unmanageable.*

15. Advantages Of C++

- **Extremely fast** – compiled straight to machine code, no interpreter overhead.
- **Full control over memory** – critical for systems with limited resources.
- **Multi-paradigm** – procedural, OOP and generic programming all in one language.
- **Massive existing codebases** – huge amounts of performance-critical infrastructure software is already written in C++, so it isn't going anywhere.
- **Portable across platforms** – recompile the same source for Windows/macOS/Linux.
- **Rich Standard Library (STL)** – ready-made, well-tested containers and algorithms.
- **Great for performance-critical fields** – games, finance, embedded, competitive programming.

...and the trade-offs (so you have the full picture)

- **Steeper learning curve** – pointers, memory management and manual typing take longer to master than Python's simpler model.
- **Slower to develop in** – you write more code, and must recompile after every change.
- **Easy to introduce memory bugs** – memory leaks and dangling pointers don't exist in memory-managed languages like Python or Java.
- **Less beginner tooling** – no single official installer; you assemble your own compiler + editor + build system.

Bottom line: C++ trades a gentler learning curve for raw speed and control – which is exactly why it still runs the game engines, operating systems and trading systems where every millisecond and every byte of memory counts.

16. A Few Extra Things Worth Knowing

Conditionals: if / else if / else

```
int age = 18;
if(age < 13){
    cout << "child";
} else if(age < 20){
    cout << "teenager";
} else{
    cout << "adult";
}
```

Compiled vs Interpreted – the key difference

<i>Compiled (C++)</i>	<i>Interpreted (e.g. Python)</i>
<i>Whole program converted to machine code BEFORE running</i>	<i>Read and run line-by-line as it goes</i>
<i>Faster to run</i>	<i>Slower to run, faster to write/test</i>
<i>Must rebuild after every change</i>	<i>Just hit run – instant feedback</i>

const – promise not to change something

```
const double PI = 3.14159; // trying to change PI later is now a compile error
```

Use it when a value should never change, like a mathematical constant or a fixed setting – it lets the compiler catch accidental changes for you.

struct vs class

A `struct` is almost identical to a `class` in C++ – the only difference is a struct's members are `public` by default, while a class's are `private` by default. Structs are usually used for simple data bundles with no behaviour.

Naming rules recap

- Variables/functions: `camelCase`
- Classes: `PascalCase`
- Constants/macros: `ALL_CAPS`

Where to go next: once you're comfortable with everything in this notebook, dig deeper into the STL (vectors, maps, algorithms), then inheritance/polymorphism, then smart pointers, then pick ONE mini project (a simple bank account simulator, a tic-tac-toe game, a small library system) and build it start to finish.



Keep Building!

Every professional C++ developer once stared at a confusing compiler error for the first time too. Compile a little every day, embrace the errors as clues, and it will click.

Phumeh Mjoli

<https://uphumeh.netlify.app/Portfolio>